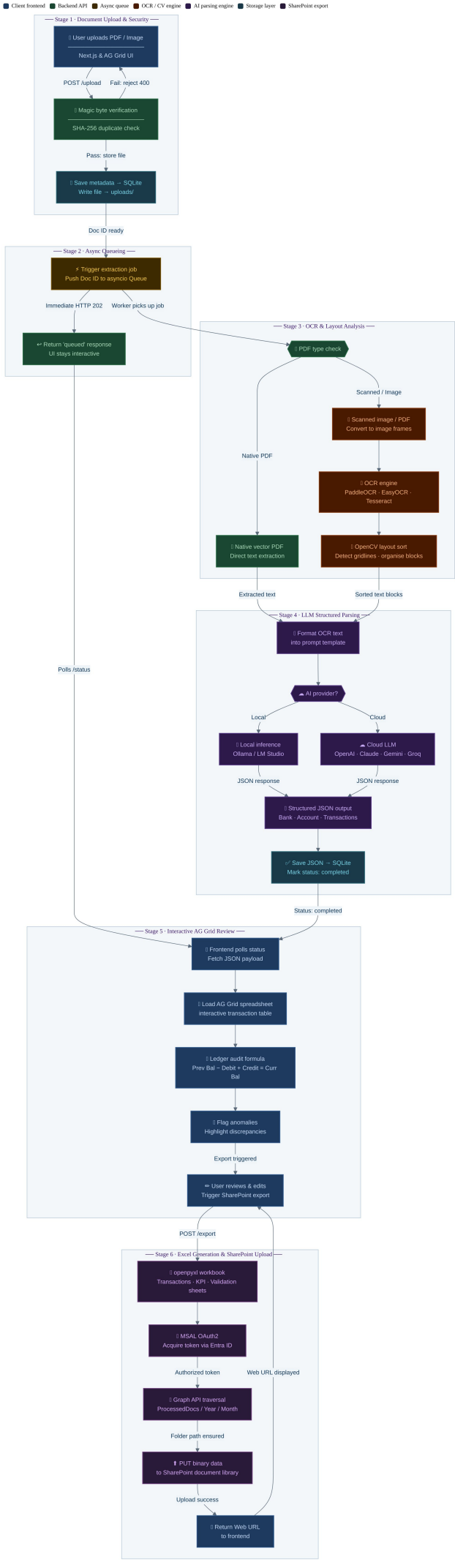

Client frontend
Backend API
Async queue
OCR / CV engine
AI parsing engine
Storage layer
SharePoint export
Stage 1 · Document Upload & Security
User uploads PDF / Image
Next.js & AG Grid UI
POST /upload
Fail: reject 400
Magic byte verification
SHA-256 duplicate check
Pass: store file
Save metadata → SQLite
Write file → uploads/
Doc ID ready
Stage 2 · Async Queueing
Trigger extraction job
Push Doc ID to asyncio Queue
Immediate HTTP 202
Worker picks up job
Return 'queued' response
UI stays interactive
Stage 3 · OCR & Layout Analysis
PDF type check
Scanned / Image
Scanned image / PDF
Convert to image frames
Native PDF
OCR engine
PaddleOCR · EasyOCR · Tesseract
Native vector PDF
Direct text extraction
OpenCV layout sort
Detect gridlines · organise blocks
Extracted text
Sorted text blocks
Stage 4 · LLM Structured Parsing
Format OCR text
into prompt template
Polls /status
AI provider?
Local
Cloud
Local inference
Ollama / LM Studio
Cloud LLM
OpenAI · Claude · Gemini · Groq
JSON response
JSON response
Structured JSON output
Bank · Account · Transactions
Save JSON → SQLite
Mark status: completed
Status: completed
Stage 5 · Interactive AG Grid Review
Frontend polls status
Fetch JSON payload
Load AG Grid spreadsheet
interactive transaction table
Ledger audit formula
Prev Bal − Debit + Credit = Curr Bal
Flag anomalies
Highlight discrepancies
Export triggered
User reviews & edits
Trigger SharePoint export
POST /export
Stage 6 · Excel Generation & SharePoint Upload
openpyxl workbook
Transactions · KPI · Validation sheets
MSAL OAuth2
Acquire token via Entra ID
Authorized token
Web URL displayed
Graph API traversal
ProcessedDocs / Year / Month
Folder path ensured
PUT binary data
to SharePoint document library
Upload success
Return Web URL
to frontend